

Lab 5 - Differential Privacy 2

In the previous lab session, we have learned how to build a general understanding of Differential Privacy in the context of a database query. We have learned whether a database query over such a small database is differentially private or not and implemented a local differential privacy to defend against differential attacks. In this lab session, we introduce the concept of global (centralized) differential privacy that adds noise to the query result rather than adding noise to the individual data points. Have a try:)

Objectives

1. Create a simple database and generate parallel databases
2. Create query functions and calculate sensitivity
3. Attempt to complete several differential attacks (Review Lab 3)
4. Implement **global (centralized) differential privacy**

1 Preparation

1.1 Step 1: create a simple database

```
[1]: import numpy as np

# the number of entries in our database
num_entries = 1000

db = np.random.random_sample(num_entries) > 0.5
db

[1]: array([False,  True,  True,  True,  True, False,  True,  True, False,
          False, False,  True,  True, False, False, False,  True,  True,
           True, False, False,  True, False,  True, False,  True, False,
          False,  True, False,  True,  True, False, False, False,  True,
          False,  True,  True,  True, False, False, False,  True, False,
          False, False,  True,  True, False, False,  True, False,  True,
           True, False,  True,  True,  True,  True, False, False,  True,
          False,  True,  True, False, False,  True,  True, False,  True,
           True,  True, False,  True,  True,  True, False, False, False,
          False,  True, False, False, False,  True,  True,  True, False,
```

[illegible]


```

True, False, True, True, False, False, True, False, True,
False, False, False, True, False, True, False, False, False,
False, True, False, True, True, False, True, True, False,
True, True, False, False, True, True, False, True, True,
True, True, True, False, True, True, False, False, True,
True, True, True, False, False, False, False, False, True,
False])

```

1.2 Step 2: generate parrallel databases

```

[2]: def get_parrallel_db(db, remove_index):
      pdb = list(db)
      # remove the entry by index
      del pdb[remove_index]
      return np.array(pdb)

```

```

[3]: get_parrallel_db(db, 2) # generate a parralleb database by removing the 2nd
      ↪ entry in the intial database
print('Size of intial db:', len(db))
print('Size of parrallel db:', len(get_parrallel_db(db, 2)))

```

Size of intial db: 1000
Size of parrallel db: 999

```

[4]: def get_parrallel_dbs(db):
      parrallel_dbs = list()
      for i in range(len(db)):
          # each parrallel database removes the ith entry from the intial database
          pdb = get_parrallel_db(db, i)
          parrallel_dbs.append(pdb)
      return parrallel_dbs

```

```

[5]: def create_db_and_parrallels(num_entries):
      db = np.random.random_sample(num_entries) > 0.5
      pddb = get_parrallel_dbs(db)
      return db, pddb

```

1.3 Step 3: calculate the sensitivity in terms of query functions

```

[6]: def query_sum(db):
      return db.sum()

```

```

[7]: def sensitivity_sum(n_entries):
      # generate the initial database and all the possible parrallel databases

```

```

db, pobjs = create_db_and_parallelisms(n_entries)

# sum value of the initial database
full_db_result = query_sum(db)

maximum_distance = 0
for pdb in pobjs:
    # sum value of each parallel database
    pdb_result = query_sum(pdb)

    # the difference between the sum values of the initial and each
    ↪parallel database
    db_distance = np.abs(pdb_result-full_db_result)
    # find out and return the maximum difference from all those differences
    if(db_distance > maximum_distance):
        maximum_distance = db_distance
return maximum_distance

```

```
[8]: sensitivity_sum(1000)
```

```
[8]: 1
```

```
[9]: # calculate the mean value of the entries in a dataset
def query_mean(db):
    return db.mean()

```

```
[10]: def sensitivity_mean(n_entries):
    # generate the initial database and all the possible parallel databases
    db, pobjs = create_db_and_parallelisms(n_entries)

    # mean value of the initial database
    full_db_result = query_mean(db)

    maximum_distance = 0
    for pdb in pobjs:
        # mean value of each parallel database
        pdb_result = query_mean(pdb)

        # the difference between the mean values of the initial and each
        ↪parallel database
        db_distance = np.abs(pdb_result-full_db_result)

        # find out and return the maximum difference from all those differences
        if(db_distance > maximum_distance):
            maximum_distance = db_distance
    return maximum_distance

```

```
[11]: sensitivity_mean(1000)
```

```
[11]: 0.0005205205205204844
```

```
[12]: def query_threshold(db, threshold=5):  
    if db.sum()>threshold:  
        return 1  
    else:  
        return 0
```

```
[13]: def sensitivity_threshold(n_entries):  
    # generate the initial database and all the possible parrallel databases  
    db, pddb = create_db_and_parrallels(n_entries)  
  
    # boolean value if the sum value of the initial database is greater than  
    ↪ the threshold  
    full_db_result = query_threshold(db)  
  
    maximum_distance = 0  
    for pddb in pddb:  
        # boolean value if the sum value of each parrallel database is greater  
        ↪ than the threshold  
        pddb_result = query_threshold(pddb)  
  
        # the difference between the boolean values of the initial and each  
        ↪ parrallel database  
        db_distance = np.abs(pddb_result-full_db_result)  
  
        # find out and return the maximum difference from all those differences  
        if(db_distance > maximum_distance):  
            maximum_distance = db_distance  
    return maximum_distance
```

```
[14]: for i in range(10):  
    print(sensitivity_threshold(10))
```

```
0  
1  
0  
0  
0  
0  
0  
0  
0  
0  
1
```

1.4 Step 4: define a differential attack

```
[15]: db,_ = create_db_and_parrallels(100)
```

For example, let us perform a differential attack on **Row 10**. So let us create a parallel database where just the **Row 10** is missing.

```
[16]: pdb = get_parallel_db(db, remove_index=10)
```

Let us check what is the value of **Row 10** in the initial database.

```
[17]: db[10]
```

```
[17]: False
```

```
[18]: # differential attack using sum query
      sum(db) - sum(pdb)
```

```
[18]: 0
```

From the sum query attack, we can know that if Row 10 is True (1), then the sum value is 1, otherwise is 0.

```
[19]: # differential attack using mean query
      sum(db)/len(db) - sum(pdb)/len(pdb)
```

```
[19]: -0.005151515151515129
```

From the mean query attack, we can know that if Row 10 is True (1), then the mean value is none-zero, otherwise is 0.

```
[20]: # differential attack using threshold
      threshold = sum(db) - 1
      sum(pdb) > threshold
```

```
[20]: True
```

1.5 Step 5: try global (centralized) differential privacy (major)

So the idea of global differential privacy is to add noise to the query result rather than adding noise to the individual data points. In other words, it gives a noise to the query result of the entire database. The formal definition of differential noise is as below:

1.5.1 Epsilon ϵ

Let's unpack the intuition of this for a moment.

Epsilon Zero: If a query satisfied this inequality where epsilon was set to 0, then that would mean that the query for all parallel databases outputted the exact same value as the full database. As you may remember, when we calculated the “threshold” function, often the Sensitivity was 0. In that case, the epsilon also happened to be zero.

Epsilon One: If a query satisfied this inequality with epsilon 1, then the maximum distance between all queries would be 1 - or more precisely - the maximum distance between the two random distributions $M(x)$ and $M(y)$ is 1 (because all these queries have some amount of randomness in them, just like we observed in the last section).

1.5.2 Delta δ

Delta is basically the probability that epsilon breaks. Namely, sometimes the epsilon is different for some queries than it is for others. For example, you may remember when we were calculating the sensitivity of threshold, most of the time sensitivity was 0 but sometimes it was 1. Thus, we could calculate this as “epsilon zero but non-zero delta” which would say that epsilon is perfect except for some probability of the time when it’s arbitrarily higher. Note that this expression doesn’t represent the full tradeoff between epsilon and delta.

Now, we’re going to learn about how to take a query and add varying amounts of noise so that it satisfies a certain degree of differential privacy. In particular, we’re going to leave behind the Local Differential privacy previously discussed and instead opt to focus on Global differential privacy.

So, to sum up, this lesson is about adding noise to the output of our query so that it satisfies a certain epsilon-delta differential privacy threshold.

There are two kinds of noise we can add - Gaussian Noise or Laplacian Noise. Generally speaking Laplacian is better, but both are still valid. Now to the hard question...

1.5.3 How much noise should we add?

The amount of noise necessary to add to the output of a query is a function of four things:

1. the type of noise (Exponential/Laplacian)
2. the sensitivity of the query/function
3. the desired epsilon ()
4. the desired delta ()

Thus, for each type of noise we’re adding, we have different way of calculating how much to add as a function of sensitivity, epsilon, and delta. We’re going to focus on Laplacian noise. Laplacian noise is increased/decreased according to a “scale” parameter b . We choose “ b ” based on the following formula.

$$b = \text{sensitivity}(\text{query}) / \text{epsilon}$$

In other words, if we set b to be this value, then we know that we will have a privacy leakage of $\leq \text{epsilon}$. Furthermore, the nice thing about Laplace is that it guarantees this with $\delta = 0$. There are some tunings where we can have very low epsilon where delta is non-zero, but we’ll ignore them for now.

Now let us perform a global differential privacy mechanism. First, we create a database with 100 entries and its 99 parallel database with 99 entries.

```
[21]: entries_num = 100
      db, pdirs = create_db_and_parallel(database, entries_num)
```

Then we define a more generalised function to calculate the sensitivity for all query functions.

```
[22]: def sensitivity(dp, pdirs, query):
      # query value of the initial database
      full_db_result = query(db)

      maximum_distance = 0
      for pdir in pdirs:
          # query value of each parallel database
          pdir_result = query(pdir)

          # the difference between the query values of the initial and each
          # parallel database
          db_distance = np.abs(pdir_result - full_db_result)

          # find out and return the maximum difference from all those differences
          if(db_distance > maximum_distance):
              maximum_distance = db_distance
      return maximum_distance
```

Third, let us assign a value to epsilon

```
[23]: epsilon = 0.5
```

Fourth, write down the laplacian mechanism according to its definition:

$$\beta = \frac{\text{sensitivity}}{\epsilon}$$

Moreover, you can use `numpy.laplace()` to draw samples from laplacian distribution.

```
[24]: def laplacian_mechanism(db, query, sensitivity):
      beta = sensitivity/epsilon
      noise = np.random.laplace(0, beta, 1)

      return query(db) + noise
```

```
[25]: # true query result
      print('True Sum:', query_sum(db))
      print('True Mean:', query_mean(db))
```

```
True Sum: 52
True Mean: 0.52
```

```
[26]: # query result with laplacian noise
print('Noised Sum:', laplacian_mechanism(db, query_sum, sensitivity(db, pddb, query_sum)))
print('Noised Mean:', laplacian_mechanism(db, query_mean, sensitivity(db, pddb, query_mean)))
```

Noised Sum: [51.36120173]

Noised Mean: [0.52458121]

As you can see, even though we only have 100 entries, the deviation from the true query result is still very small compared to local differential privacy. This is because that global differential privacy only adds noise to the query result which add far less noise than local differential privacy mechanism.

Then let us try another global differential privacy mechanism - exponential noise. Exponential mechanism is used for non-numerical query. For example, we can use Laplacian mechanism for the query 'how many people's answer is yes?'. But for the query such as 'what is the favourite answer ({yes, no}) for this dataset?', we should use exponential mechanism.

The definition of expoenntial_mechanism is $Pr = \frac{\epsilon * Q(D, r)}{2\Delta}$ where D is the dataset, and r is the output for D . $Q(D, r)$ can be seen as how good the output r is for D .

Again, we create a database with 100 entries and its 99 parrallel database with 99 entries.

```
[27]: entries_num = 100
db, pddb = create_db_and_parrallels(entries_num)
```

Then, we can calculate how many answers are 'yes'(True) and 'no'(False) in the true dataset db.

```
[28]: cat1, cat2 = list(db).count(True), list(db).count(False)
print(cat1, cat2)
```

50 50

So we can calculate the porpotions of these answer catagories which is can be seen as the $Q(D, r)$ in this case.

```
[29]: pro1, pro2 = cat1/len(db), cat2/len(db)
print(pro1, pro2)
```

0.5 0.5

The sensitivity is 1 because the maximun difference from yes (1) to no (0) is 1.

```
[30]: sensitivity = 1
```

```
[31]: import math
def exponential_mechanism(Q, sensitivity):
    pro = math.exp((epsilon*Q)/(2*sensitivity))

    return pro
```

```
[32]: pro1 = exponential_mechanism(pro1, sensitivity)
      pro2 = exponential_mechanism(pro2, sensitivity)
      print(pro1, pro2)
```

1.1331484530668263 1.1331484530668263

Then we normalise the porpotion to the range (0,1).

```
[33]: total = pro1 + pro2
      pro1 = pro1/total
      pro2 = pro2/total
      print('True porpotion:',cat1/len(db), cat2/len(db) )
      print('Noised porpotion:', pro1,pro2)
```

True porpotion: 0.5 0.5

Noised porpotion: 0.5 0.5

2 Quiz

- 2.0.1 1. Try different sizes (size=200, 400, ..., 2000) off datasets with the same noise (noise = 0.1) 10 times in local differential privacy, then calculate the average deviation and standard deviation and show your result to the tutor.
- 2.0.2 2. Try different noise (noise=0.1, 0.2, ..., 1) with the same size of dataset (size = 2000) 10 times in local differential privacy, then calculate the average deviation and standard deviation and show your result to the tutor.
- 2.0.3 3. Try different epsilon (0.1, 0.2, ..., 1) in Laplacian mechanism in global differential privacy, and show the result to your tutor.
- 2.0.4 4. Try different epsilon (0.1, 0.2, ..., 1) in exponential mechanism in global differential privacy, and show the result to your tutor.

[]: